

Digital Design

Sequential Logic Design

(Latch & FF)

A. Sahu

Dept of Comp. Sc. & Engg.

Indian Institute of Technology Guwahati

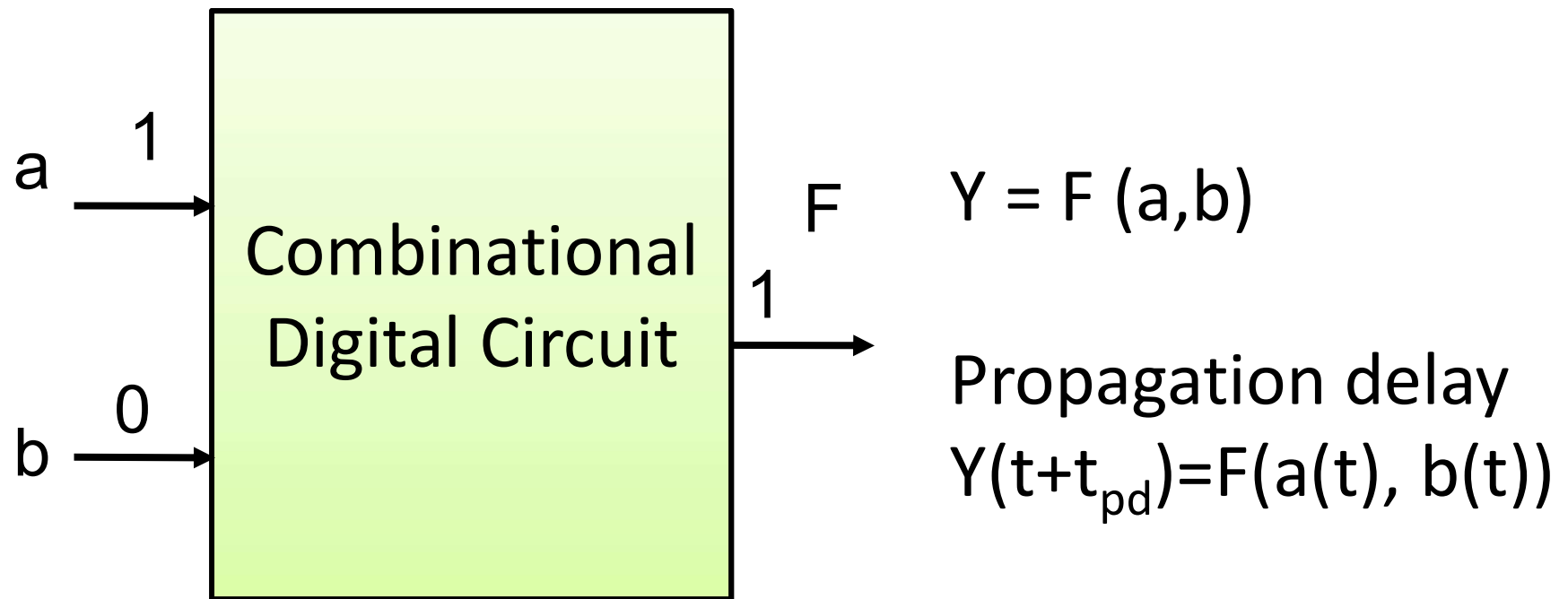
Some part of Slides Adopted from Digital System Book by Frank Vahid

Outline

- Combinational Vs Sequential Logic Design
- Design a **Circuit**, that stores one bit
 - RS latch
- Stabilizing RS latch : Level Sensitive
- Clocked Latch : Flip Flop- Edge Sensitive
- D, JK, T flip flops
- Characterization Table and Equation
 - RS, D, JK and T Flip flop

Combinational Vs Sequential Logic

- Combinational circuit
 - **Output depends on present input**
 - Examples: F (A,B,C), FA, HA, Multiplier, Decoder, Multiplexor, Adder, Priority Encoder

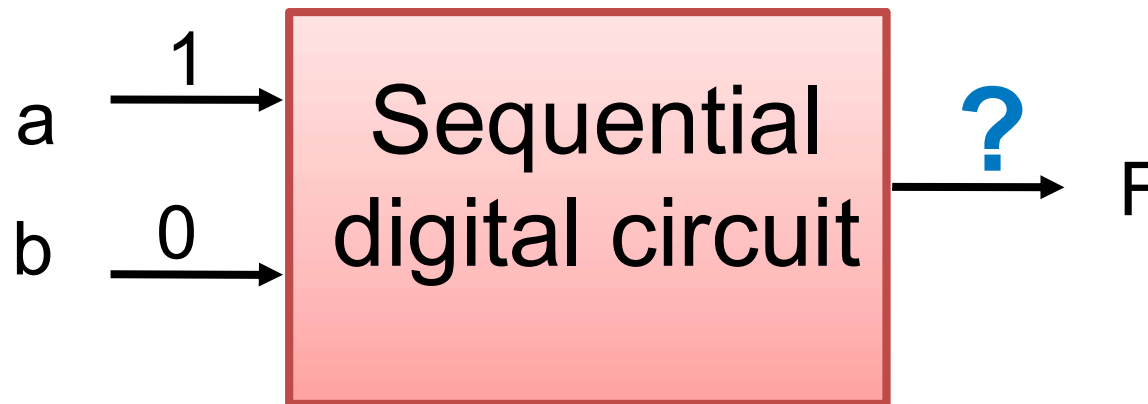


Combinational Vs Sequential Logic

- Sequential circuit
 - Output depends not just on present inputs
 - But also on past sequence of inputs (State)
 - Stores bits, also known as having “state”

Combinational Vs Sequential Logic

- Simple example: a circuit that counts up in binary

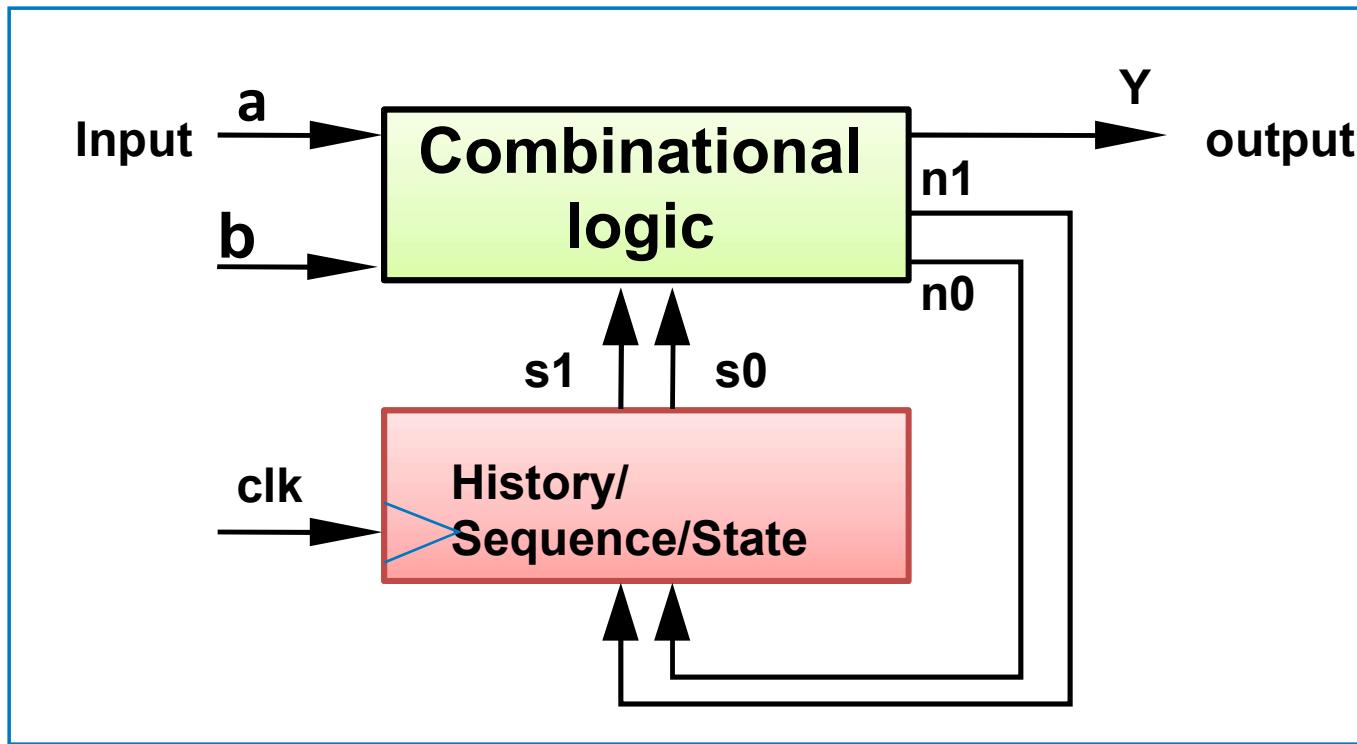


Must know sequence of past inputs to know output

$$Y(t) = F(a(t), b(t), H)$$

H is History/Sequence/State

Sequential Circuit



Where to
Store this
History

(Memory
Element)

$$Y(t) = F(a(t), b(t), H)$$

H is History/Sequence/State

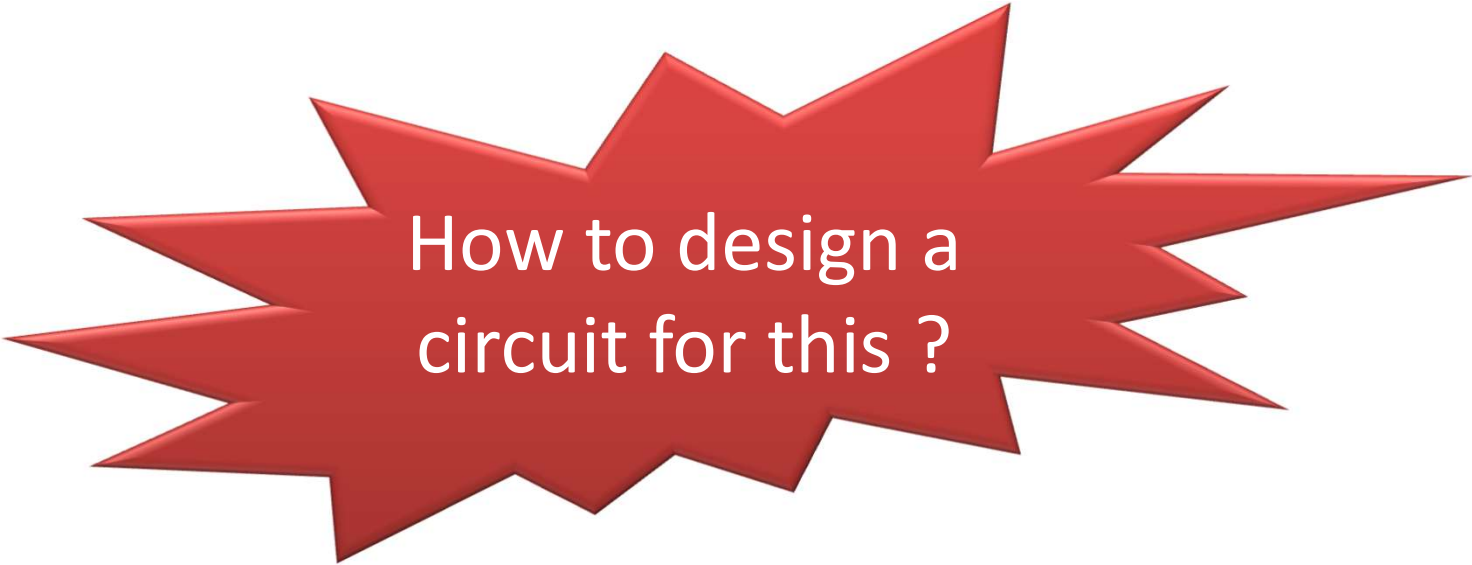
Mechanical Storage of State

- Retractable Pen
 - Pressing result a Toggle
 - On if it is off
 - Off if it is On
 - It store the state mechanically
-
- Circular wheel: saw type rotate each press
 - Alternate Pattern : one height is low other is high



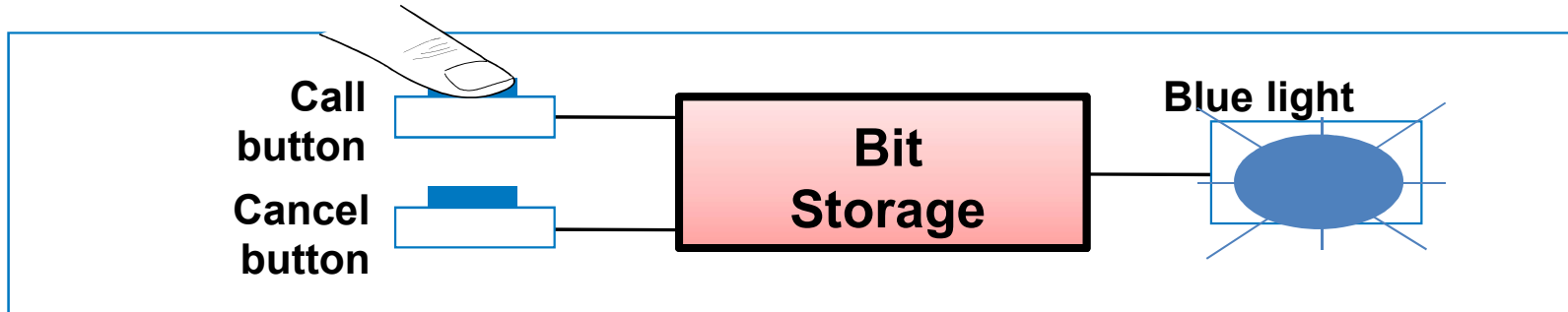
Example Needing Bit Storage

- Example: Flight attendant call button
 - Press call: light turns on
 - ***Stays on*** after button released
 - Press cancel: light turns off

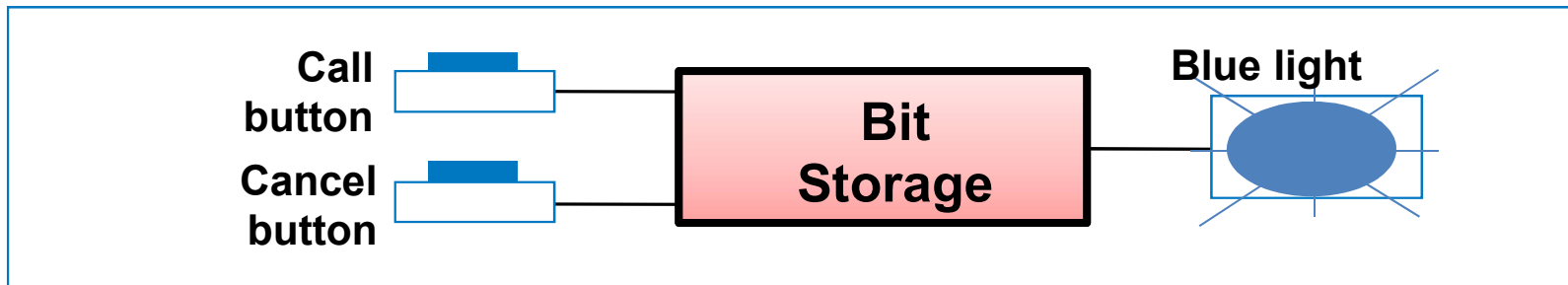


How to design a
circuit for this ?

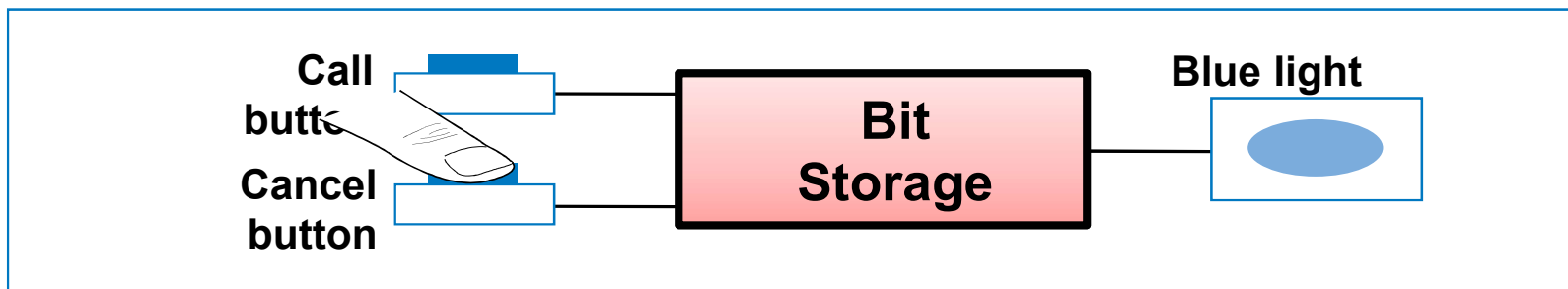
Example Needing Bit Storage



1. Call button pressed – light turns on



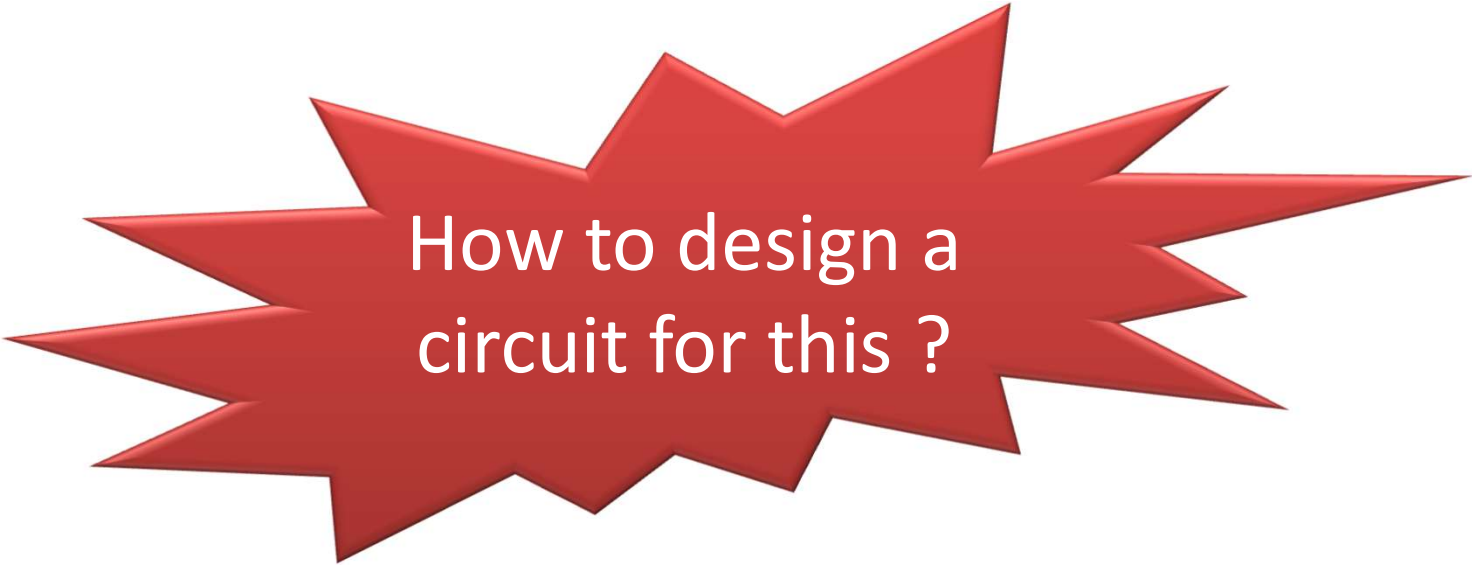
2. Call button released – light stays on



3. Cancel button pressed – light turns off

Example Needing Bit Storage

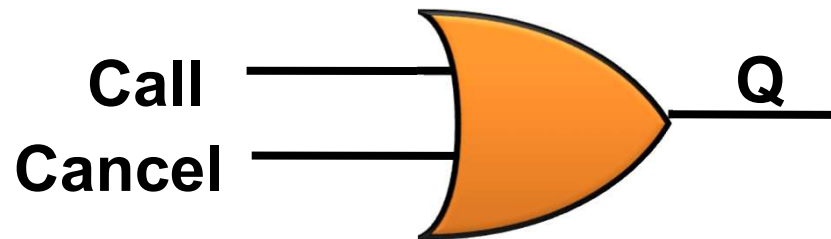
- Flight attendant call button
 - Press call: light turns on
 - ***Stays on*** after button released
 - Press cancel: light turns off



How to design a
circuit for this ?

Example Needing Bit Storage

- Flight attendant call button
 - Press call: light turns on : ***Stays on*** after button released
 - Press cancel: light turns off
- Logic gate circuit to implement this?

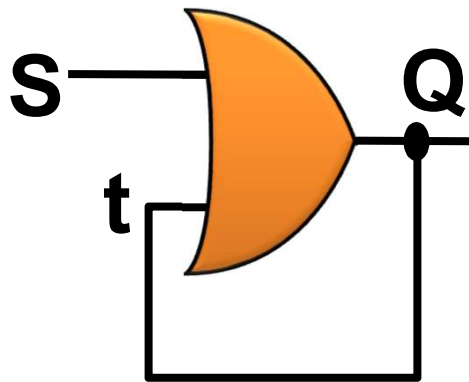


Doesn't work. $Q=1$ when $\text{Call}=1$, but doesn't stay 1 when Call returns to 0

Need some form of "feedback" in the circuit

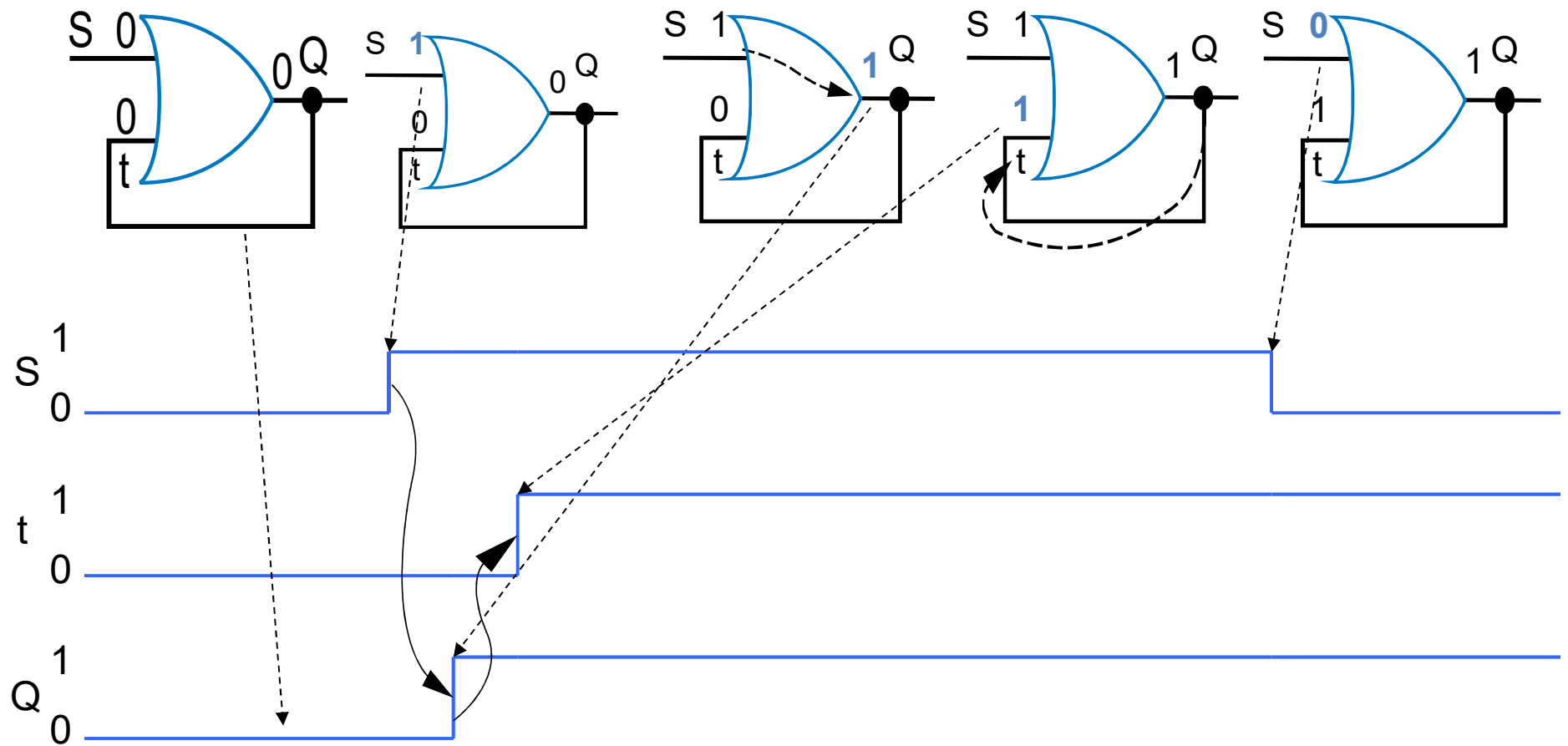
First attempt at Bit Storage

- We need some sort of feedback
 - Does circuit this circuit do what we want?



- No: Once **Q** becomes 1 (when **S**=1), **Q** stays 1 forever – no value of **S** can bring **Q** back to 0

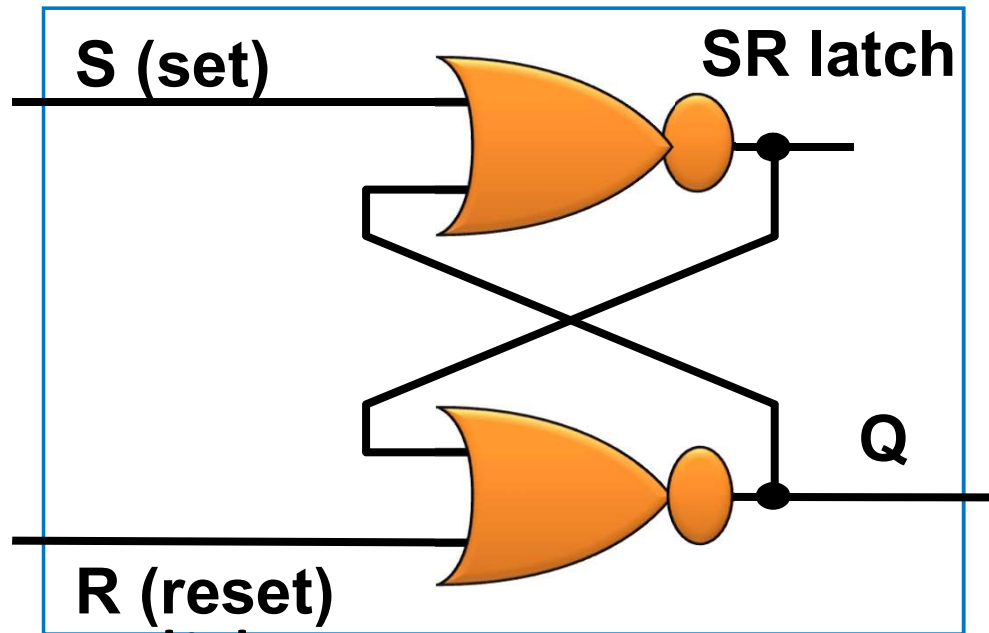
First attempt at Bit Storage



Once **Q** becomes 1 (when **S**=1), **Q** stays 1 forever – no value of **S** can bring **Q** back to 0

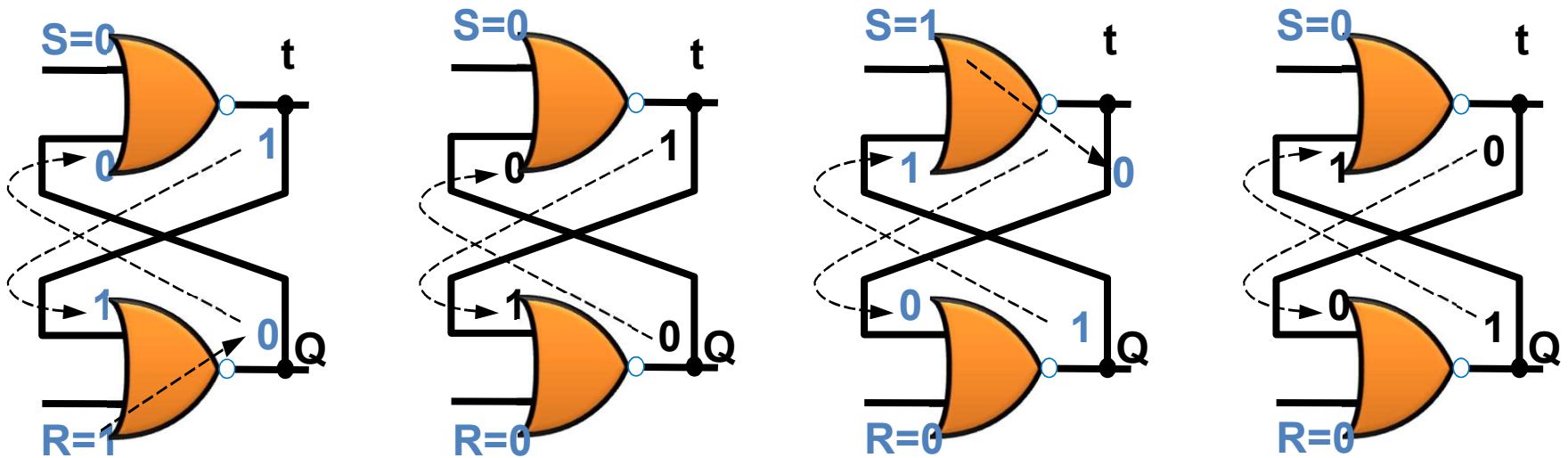
Bit Storage Using an SR Latch

- With cross-coupled NOR gates
 - Does the circuit to the right ?
 - Do what we want?



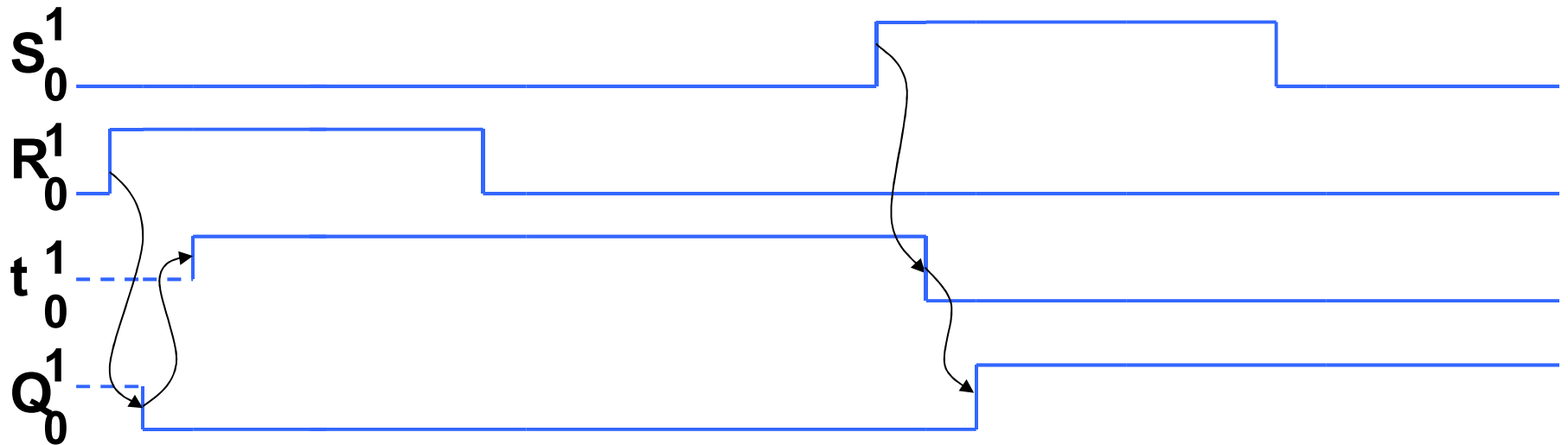
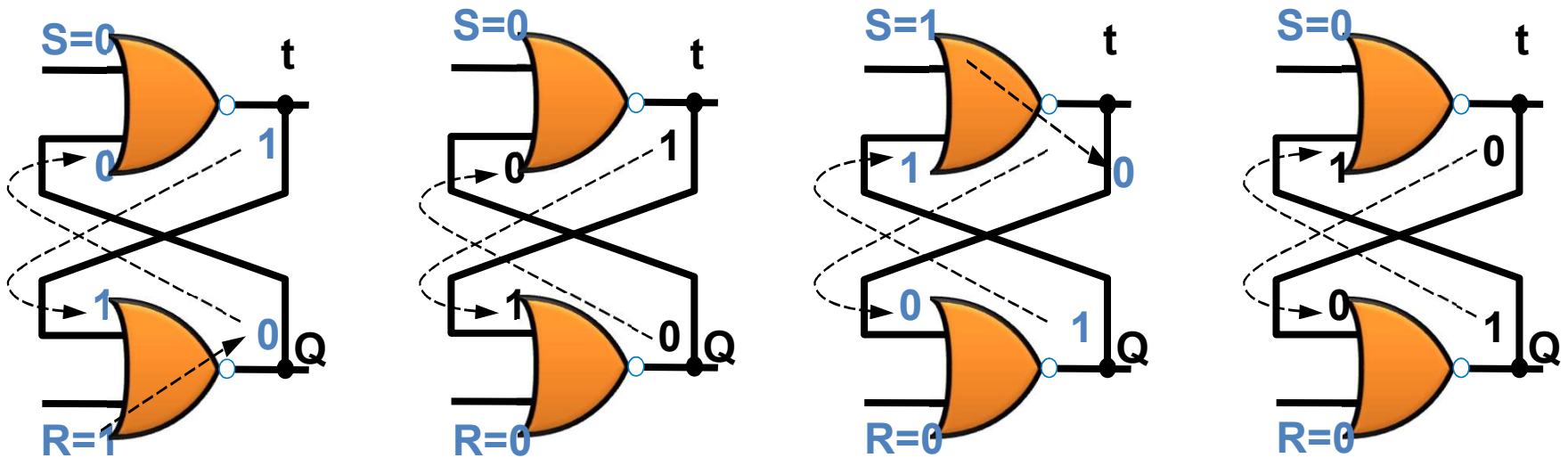
- Yes! How did someone come up with that circuit?
 - Maybe just trial and error, a bit of insight...

Bit Storage Using an SR Latch

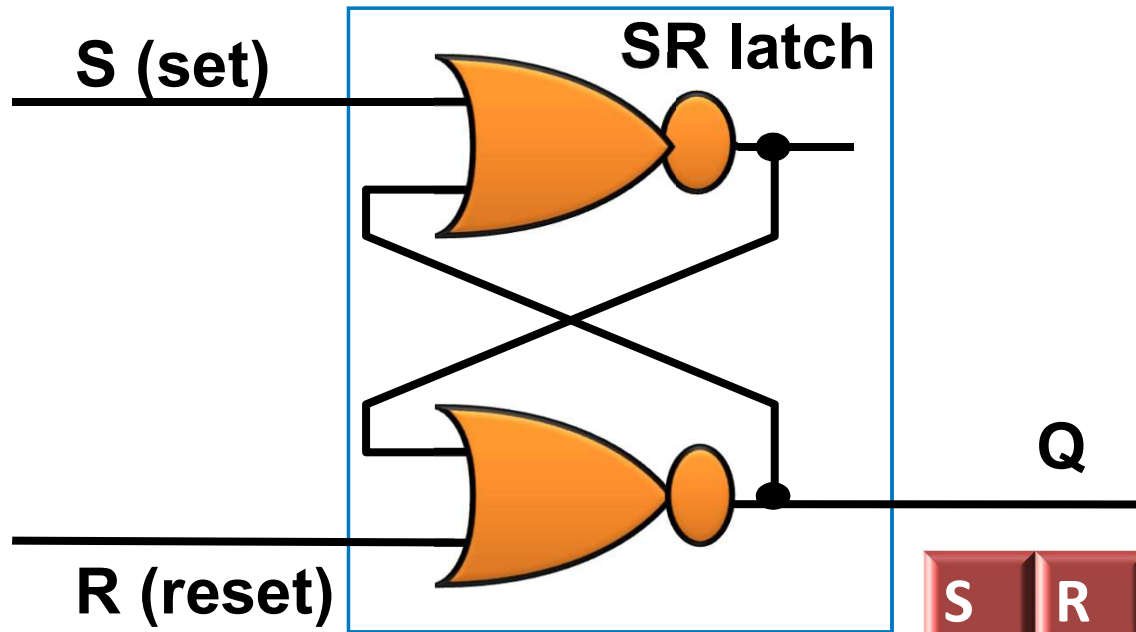


- When $R=1, S=0 \rightarrow$ As $R=1$, ensure **Q set to 0**, As $S=0$ and $Q=0$ it result $t=1$
- When $R=0, S=0 \rightarrow$ As $t=1$, **Q set to 0**, As $S=0$ and $Q=0$ it result $t=1$ (State Continue)
- When $R=0, S=1 \rightarrow$ As $S=1$, ensure **t set to 0 and Q=1**
- When $R=0, S=0 \rightarrow$ As $t=0$, **Q set to 1**, As $S=0$ and $Q=1$ it result $t=0$ (State Continue)

Bit Storage Using an SR Latch



Function Table of SR Latch



S	R	Q	Q'	
1	0	1	0	
0	0	1	0	After S=1, R=0
0	1	0	1	
0	0	0	1	After S=0, R=1
1	1	0	0	Forbidden

Set : make out put 1

Reset : make out put 0

Verilog Code for SR-Latch

```
module sr_latch(  
    input wire S, R,  
    output wire Q, Qb);  
  
    assign Q    = ~(R | Qb);  
    assign Q_b = ~(S | Q);  
  
endmodule
```

Verilog Code for SR-Latch

```
module sr_latch_tb;
reg s,r; // we can modify registers to simulate the inputs
wire q, qb; // we can't modify: assigned to our module
sr_latch latch(s, r, q, qb);
initial begin
    // for waveform analysis
    $dumpfile("sr_latch.vcd");
    //$dumpvars(s, r, q, qb);
    $dumpvars;
```

Verilog Code for SR-Latch

```
s=0; r=0; #1 // case 1 (latch w/o state)
$display("s=%b, r=%b ==> q=%b, qb=%b # undefined", s, r, q, qb);
s=0; r=1; #1 // case 2 (reset)
$display("s=%b, r=%b ==> q=%b, qb=%b # reset, so q=0", s, r, q, qb);
s=1; r=0; #1 // case 3 (set)
$display("s=%b, r=%b ==> q=%b, qb=%b # set, so q=1", s, r, q, qb);
s=0; r=0; #1 // case 4 (latch with state)
$display("s=%b, r=%b ==> q=%b, qb=%b # latch,
        so q=q (keep state)", s, r, q, qb);
s=1; r=1; #1 // case 5 (invalid state)
$display("s=%b, r=%b ==> q=%b, qb=%b # invalid state ", s, r, q, qb);
end
endmodule
```

Verilog Code for SR-Latch

//Install and run verilog code in Linux (Ubuntu22.04)

```
$sudo apt-get install verilog
```

```
$sudo apt-get install gtkwave
```

```
$iverilog sr_latch.v
```

```
$/a.out
```

```
VCD info: dumpfile sr_latch.vcd opened for output.
```

```
s=0, r=0 ==> q=x, q_not=x # should be undefined
```

```
s=0, r=1 ==> q=0, q_not=1 # reset, so q=0
```

```
s=1, r=0 ==> q=1, q_not=0 # set, so q=1
```

```
s=0, r=0 ==> q=1, q_not=0 # latch, so q=q (keep state)
```

```
s=1, r=1 ==> q=0, q_not=0 # invalid state
```

```
$gtkwave sr_latch.vcd
```

Verilog Code for SR-Latch

//Install and run verilog code in Window)

Download and install iverilog and Cygwin from

<https://bleyer.org/icarus/>

<https://www.cygwin.com/>

set path to /cygdrive/c/iverilog/bin and
/cygdrive/c/iverilog/gtkwave/bin

Or set alias in ~/.bashrc and

- alias gtkwave='/cygdrive/c/iverilog/gtkwave/bin/gtkwave'
- alias iverilog='/cygdrive/c/iverilog/bin/iverilog'
- alias vvp='/cygdrive/c/iverilog/bin/vvp'

Verilog Code for SR-Latch (Windows)

In Cygwin : in source folder

```
$iverilog sr_latch.v
```

```
$vvp a.out
```

VCD info: dumpfile sr_latch.vcd opened for output.

s=0, r=0 ==> q=x, q_not=x # should be undefined

s=0, r=1 ==> q=0, q_not=1 # reset, so q=0

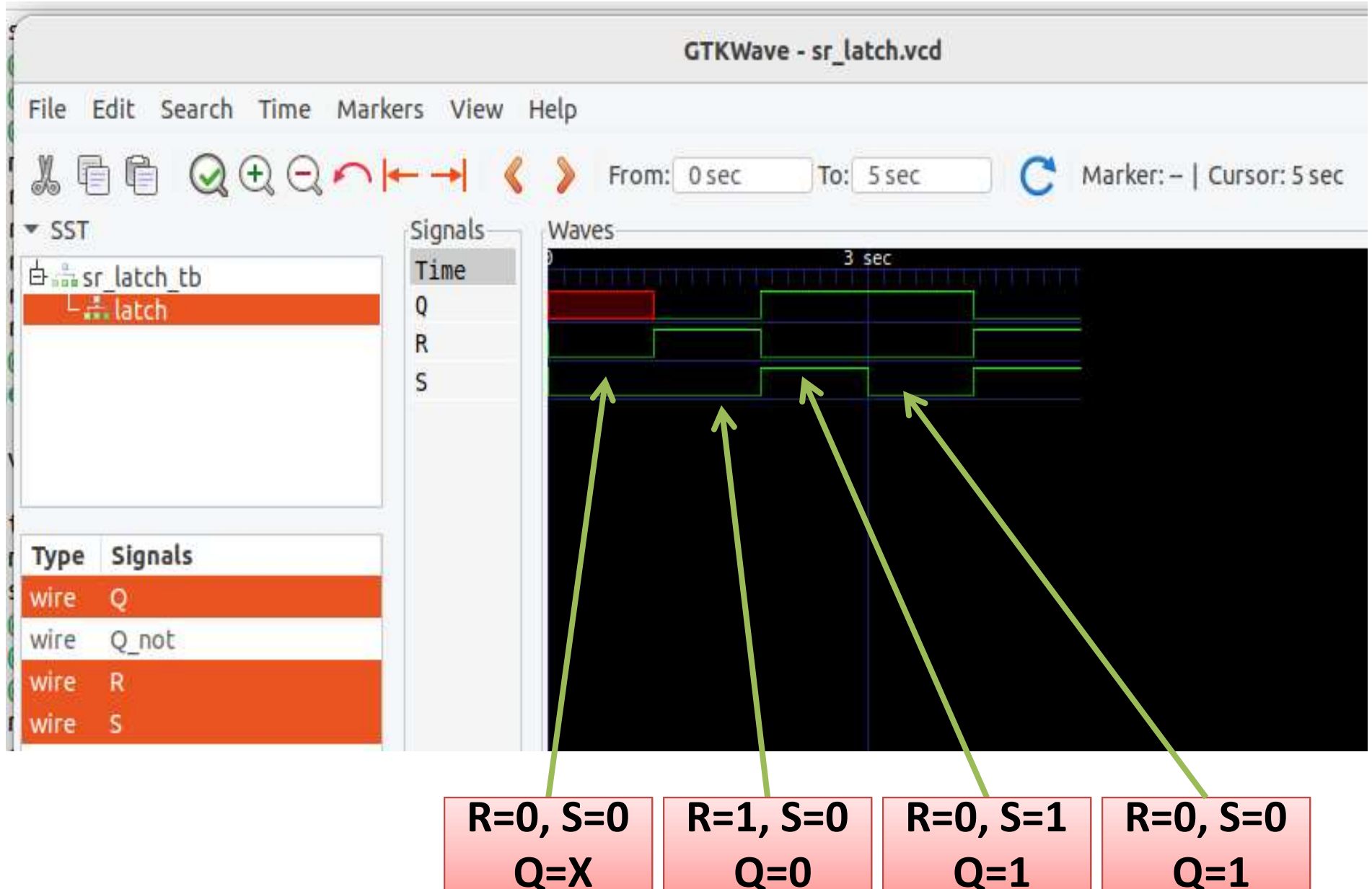
s=1, r=0 ==> q=1, q_not=0 # set, so q=1

s=0, r=0 ==> q=1, q_not=0 # latch, so q=q (keep state)

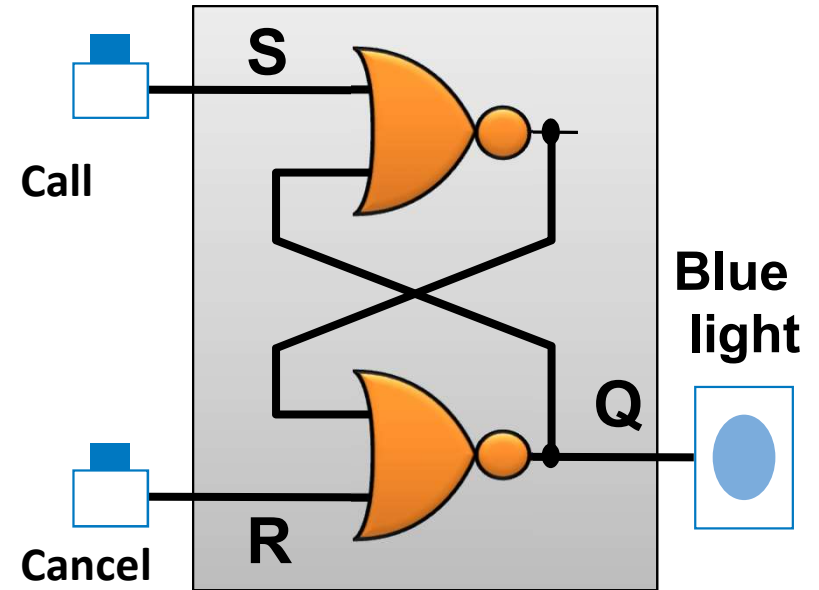
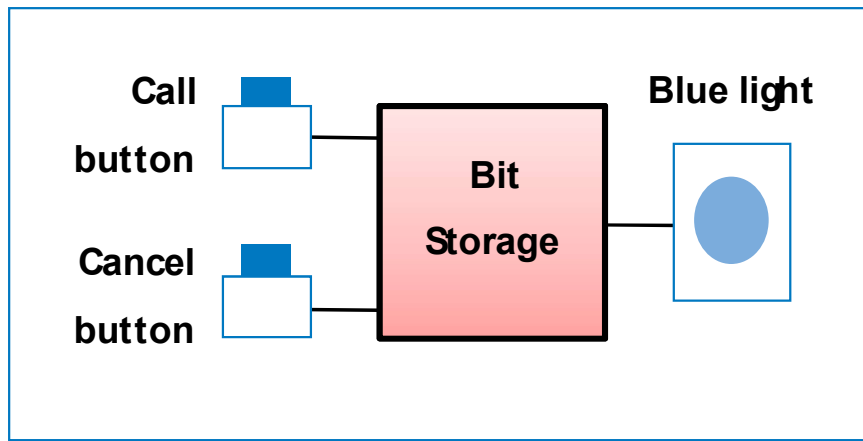
s=1, r=1 ==> q=0, q_not=0 # invalid state

```
$gtkwave sr_latch.vcd
```

Verilog Code for SR-Latch

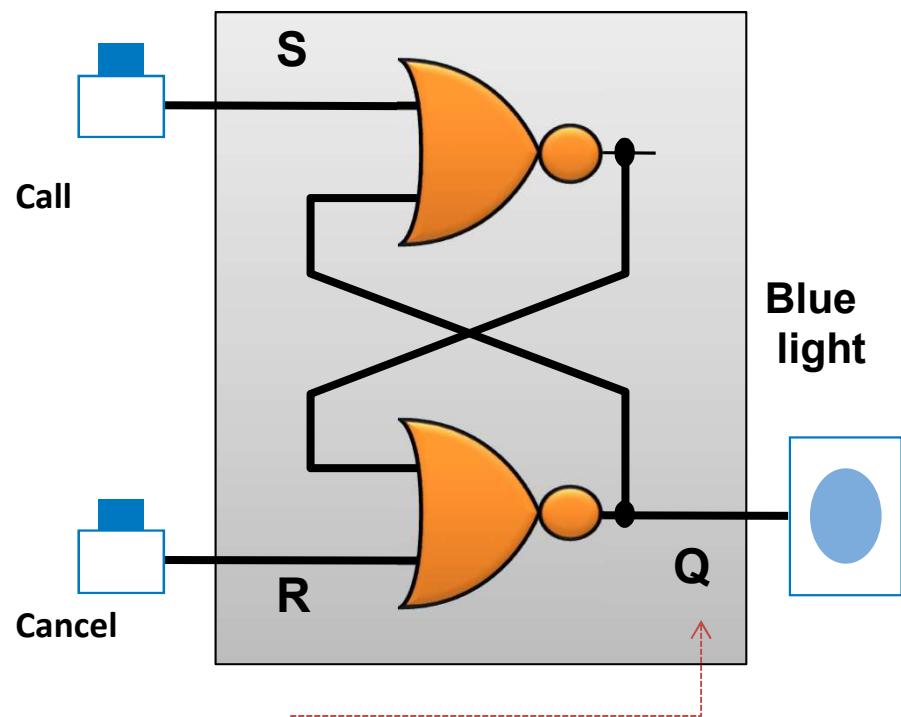
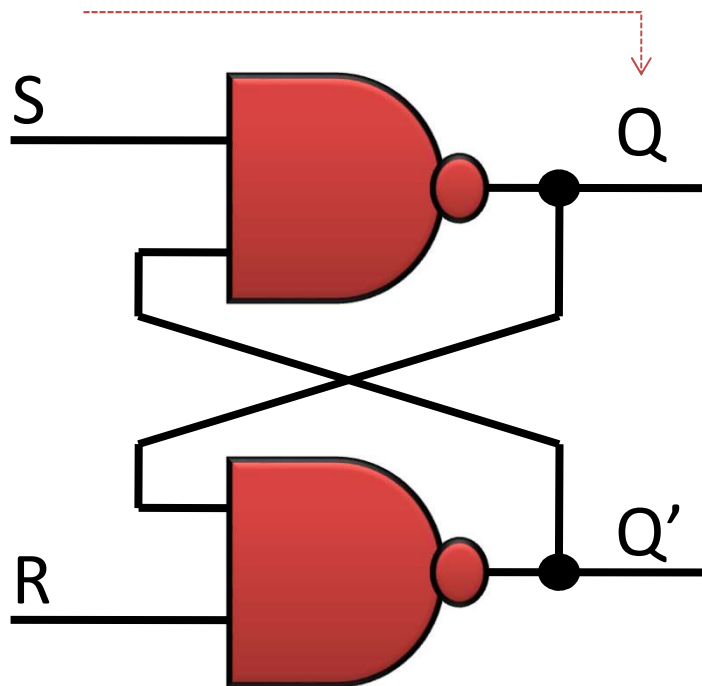


Example Using SR Latch for Bit Storage



- SR latch can serve as bit storage of flight-attendant call button : 😊 😊 😊 Great
 - Call=1 : sets Q to 1
 - Q stays 1 even after Call=0
 - Cancel=1 : resets Q to 0

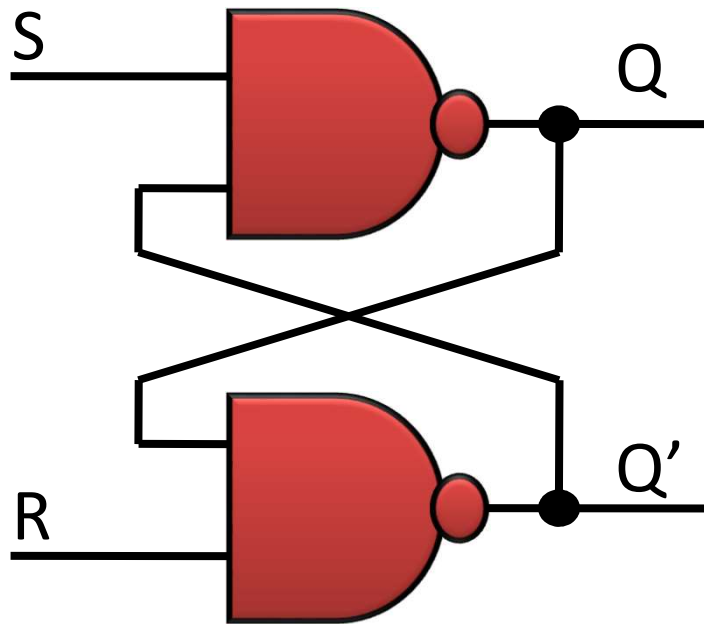
SR Latch with NAND Gates



In NOR based Latch: R was driving to Q

In NAND based Latch: S was driving to Q

SR Latch with NAND Gates

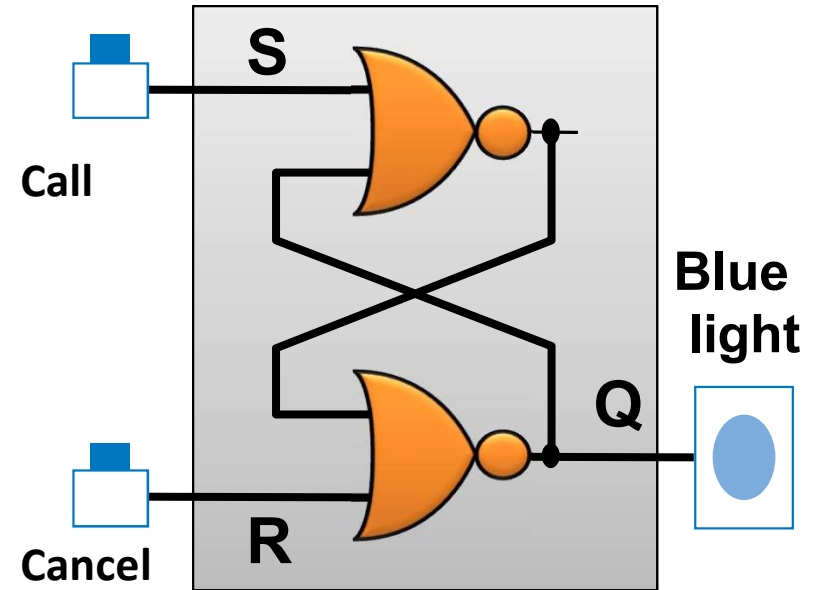
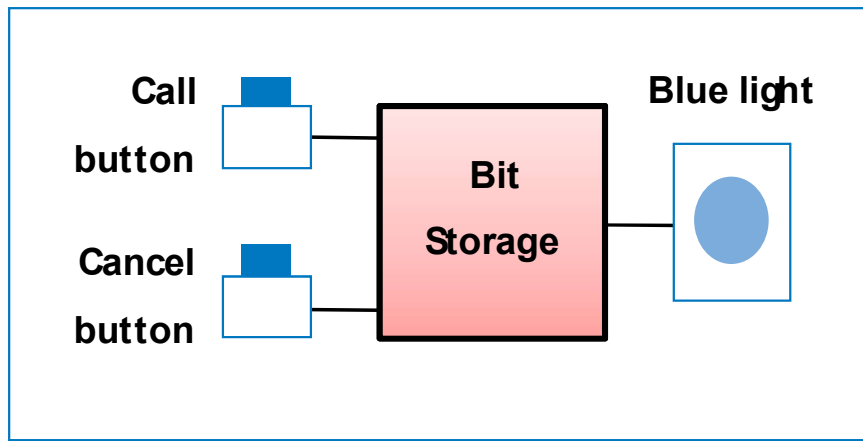


S	R	Q+
0	0	1*0* (Unpredictable)
0	1	1
1	0	0
1	1	Q

Opposite to SR Latch with NOR Gates

Set will do $Q=0$ and Reset will $Q=1$

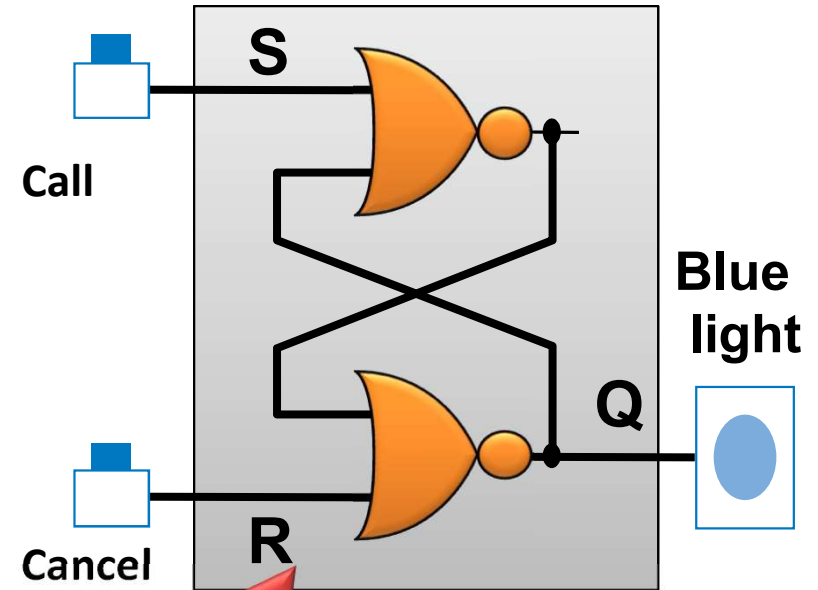
Example Using SR Latch for Bit Storage



- SR latch can serve as bit storage of flight-attendant call button : 😊 😊 😊 Great
 - Call=1 : sets Q to 1
 - Q stays 1 even after Call=0
 - Cancel=1 : resets Q to 0
- **But, there's a problem...**

Problem with SR Latch: $SR=11$

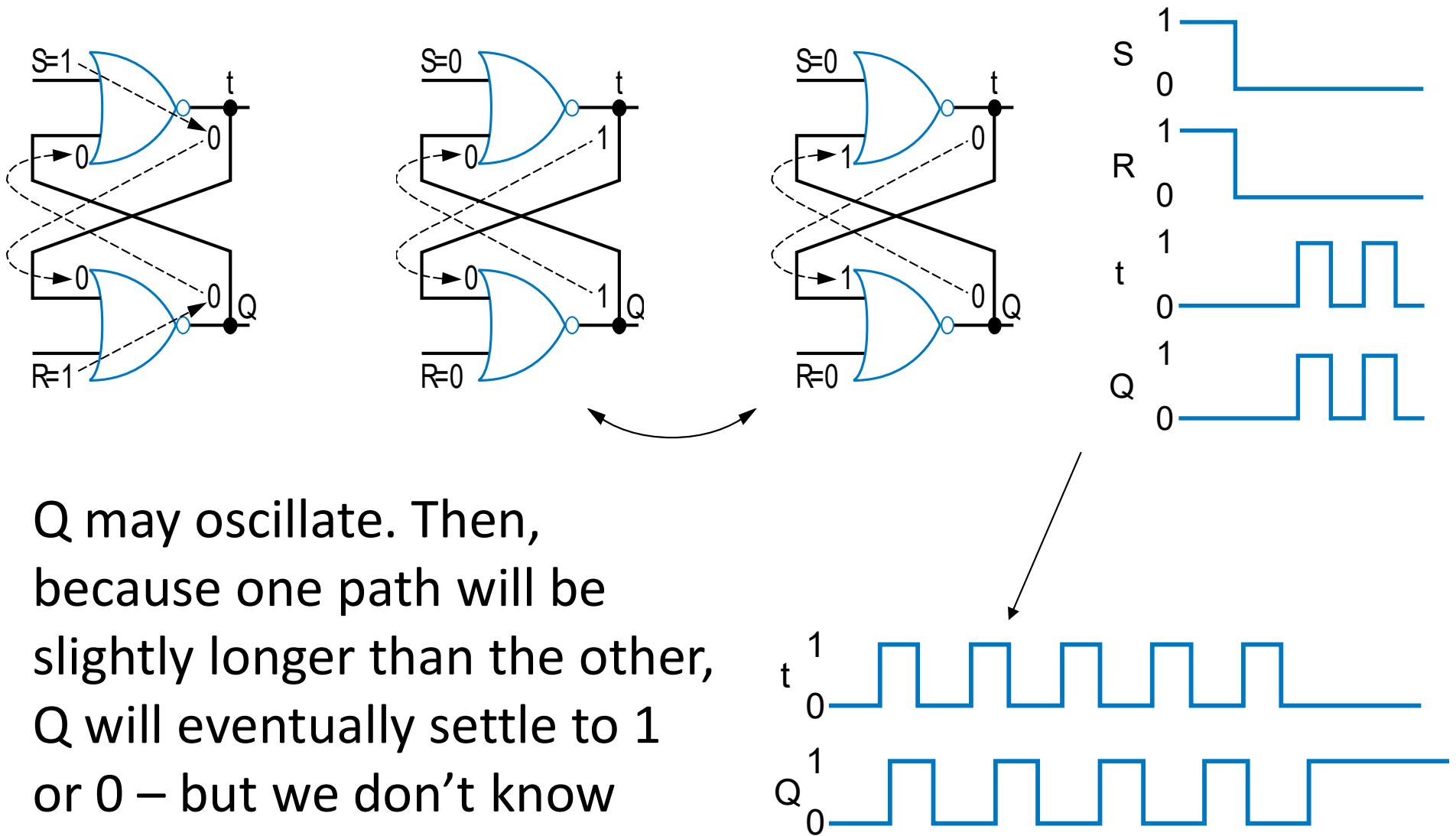
- Problem
 - If $S=1$ and $R=1$ simultaneously, we don't know what value Q will take (Unpredictable)



Race Condition :-

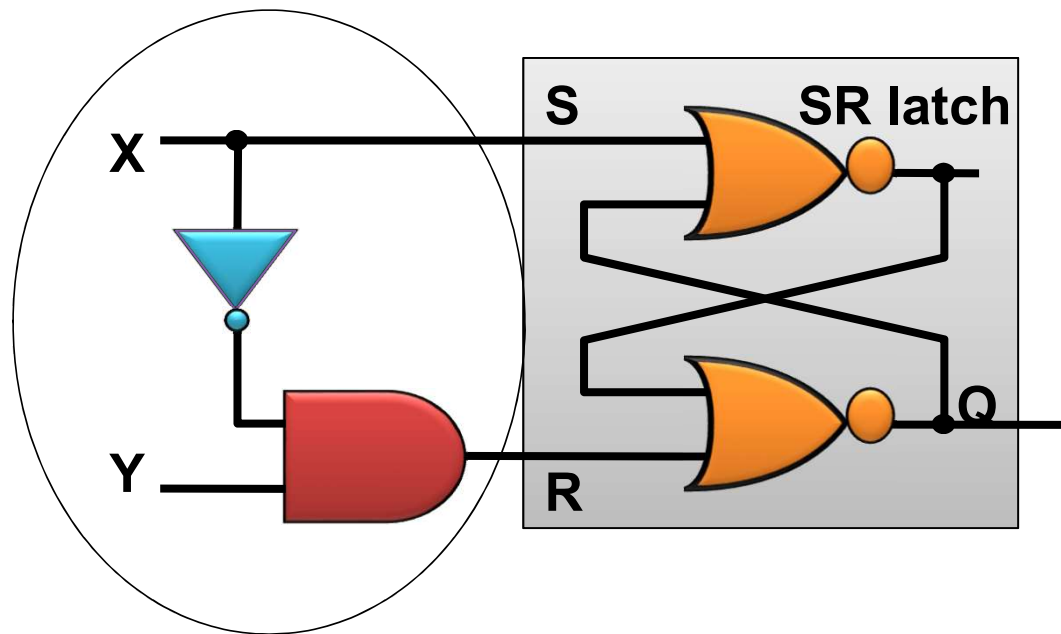
**Who will win 0 or 1?
Eventually, what will be the
value of Q?**

Problem with SR Latch



Solution to Race Condition

- We try to avoid $S=1$ and $R=1$ by the following circuit



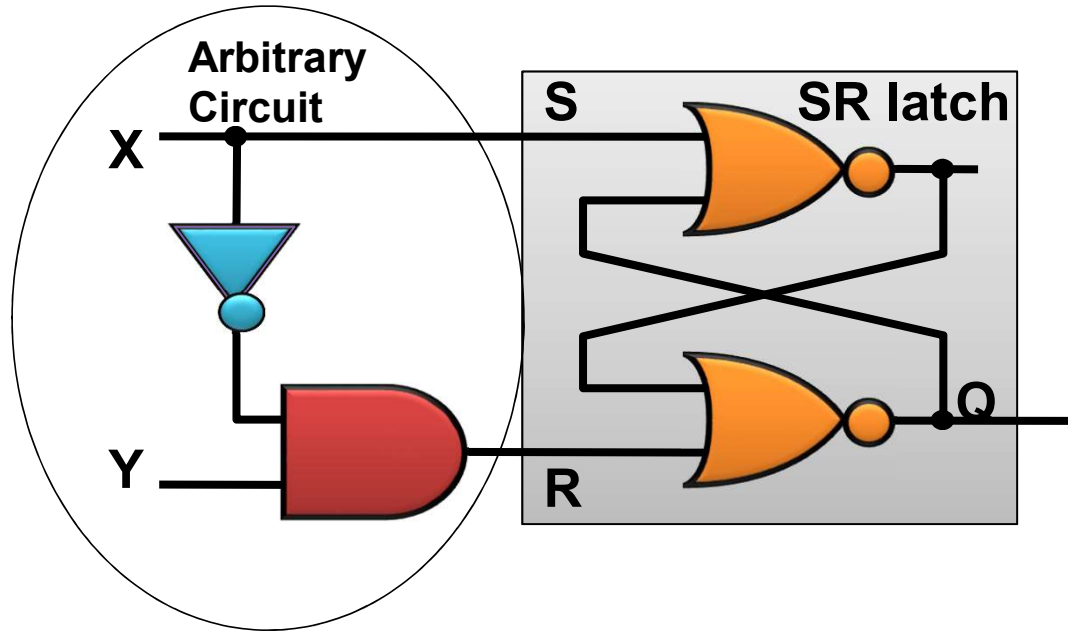
- Can we ensure value $S=1$ and $R=1$ will not happen at the same time?

Problem with SR Latch

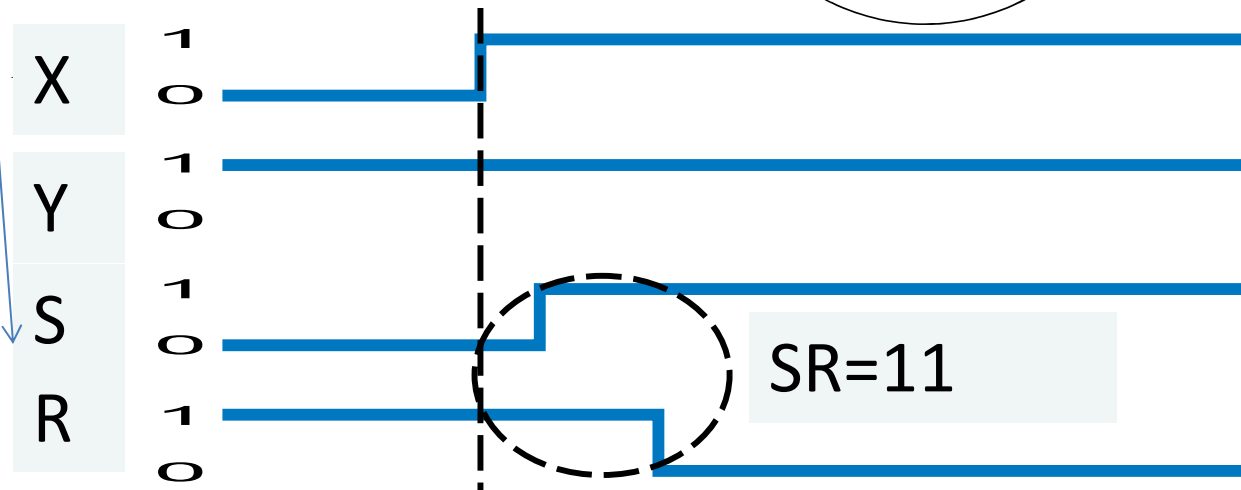
- Problem not just one of a user pressing two buttons at same time
- Can also occur even if SR inputs come from a circuit that supposedly never sets $S=1$ and $R=1$ at same time
 - But does, due to different delays of different paths

Problem with SR Latch

- $X=1$ and $Y=1$ set



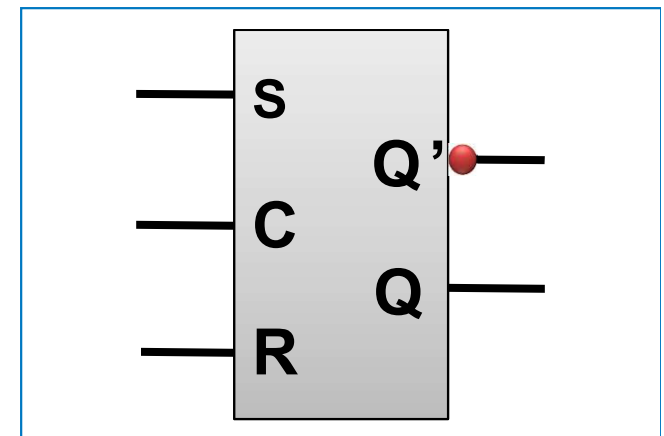
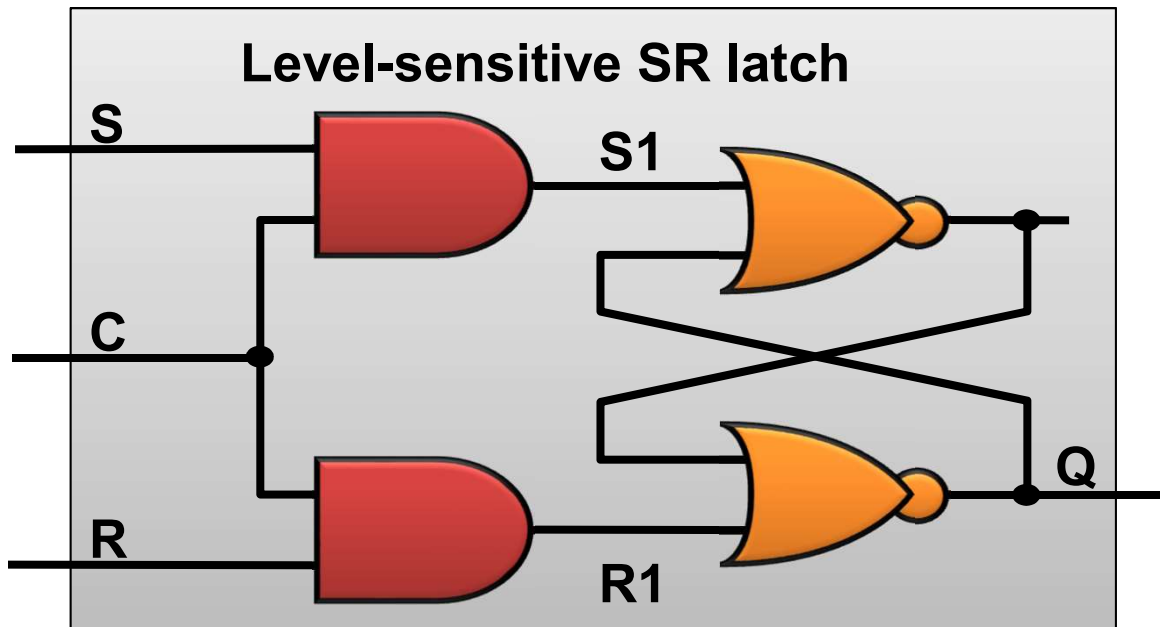
Yesterday it was
my mistake



The longer path from X to R than to S causes $SR=11$ for short time – could be long enough to cause oscillation

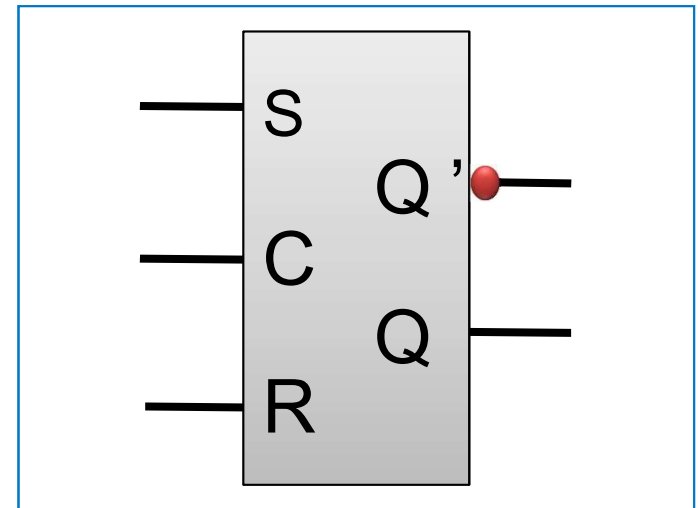
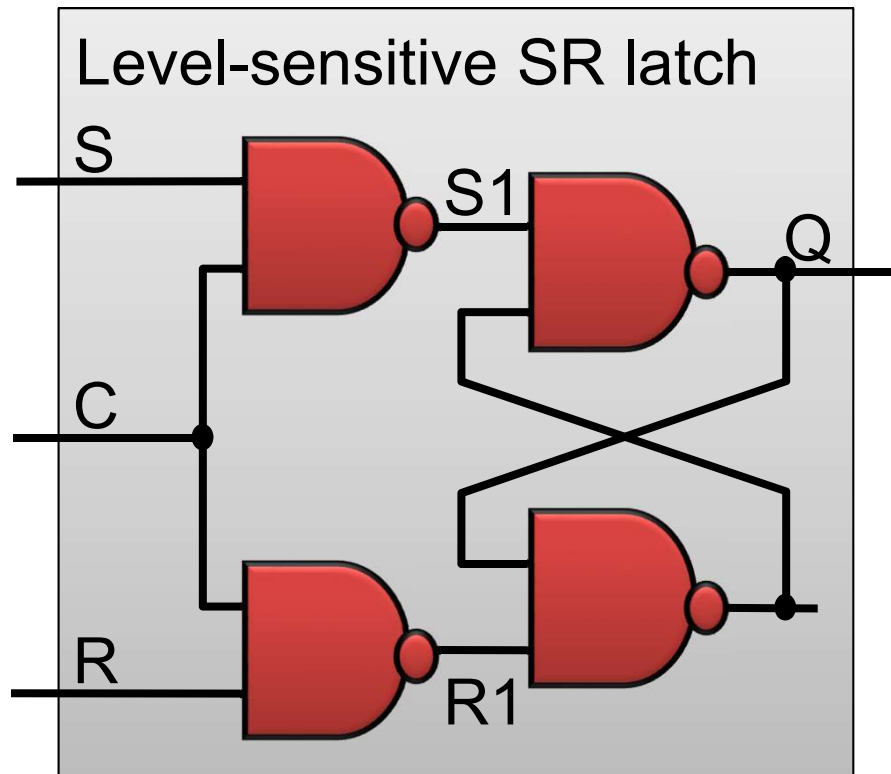
Solution: Level-Sensitive SR Latch

- Add enable input “C or En”
 - Only let S and R change when $C = 0$: Ensure circuit in front of SR never sets $SR = 11$, except briefly due to path delays
 - Change C to 1 only after sufficient time for S and R to be stable
 - When C becomes 1, the stable S and R value passes through the two AND gates to the SR latch’s S1 R1 inputs.



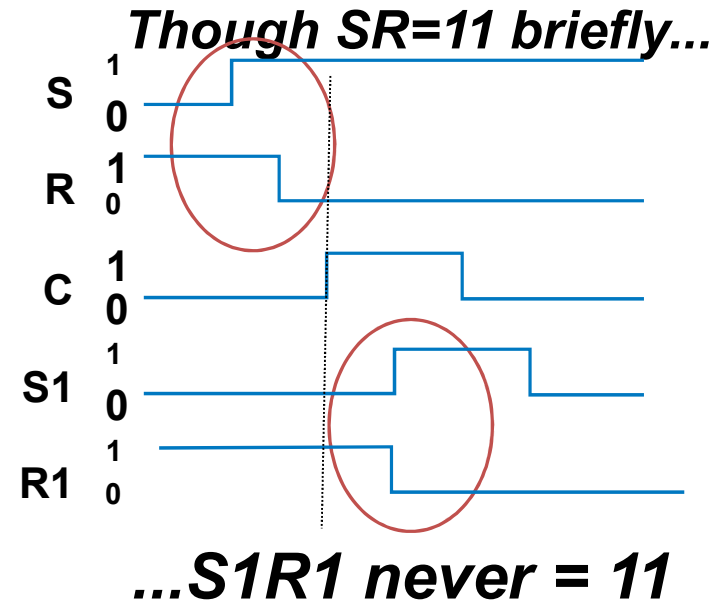
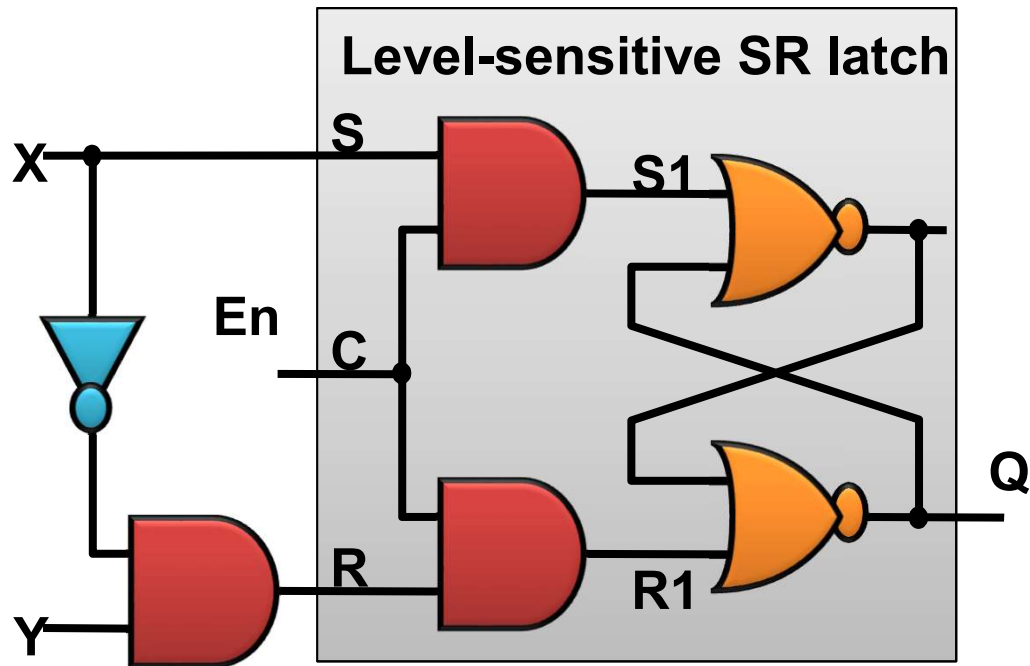
Level-sensitive SR latch symbol

Level Sensitive: SR Latch with NAND Gate

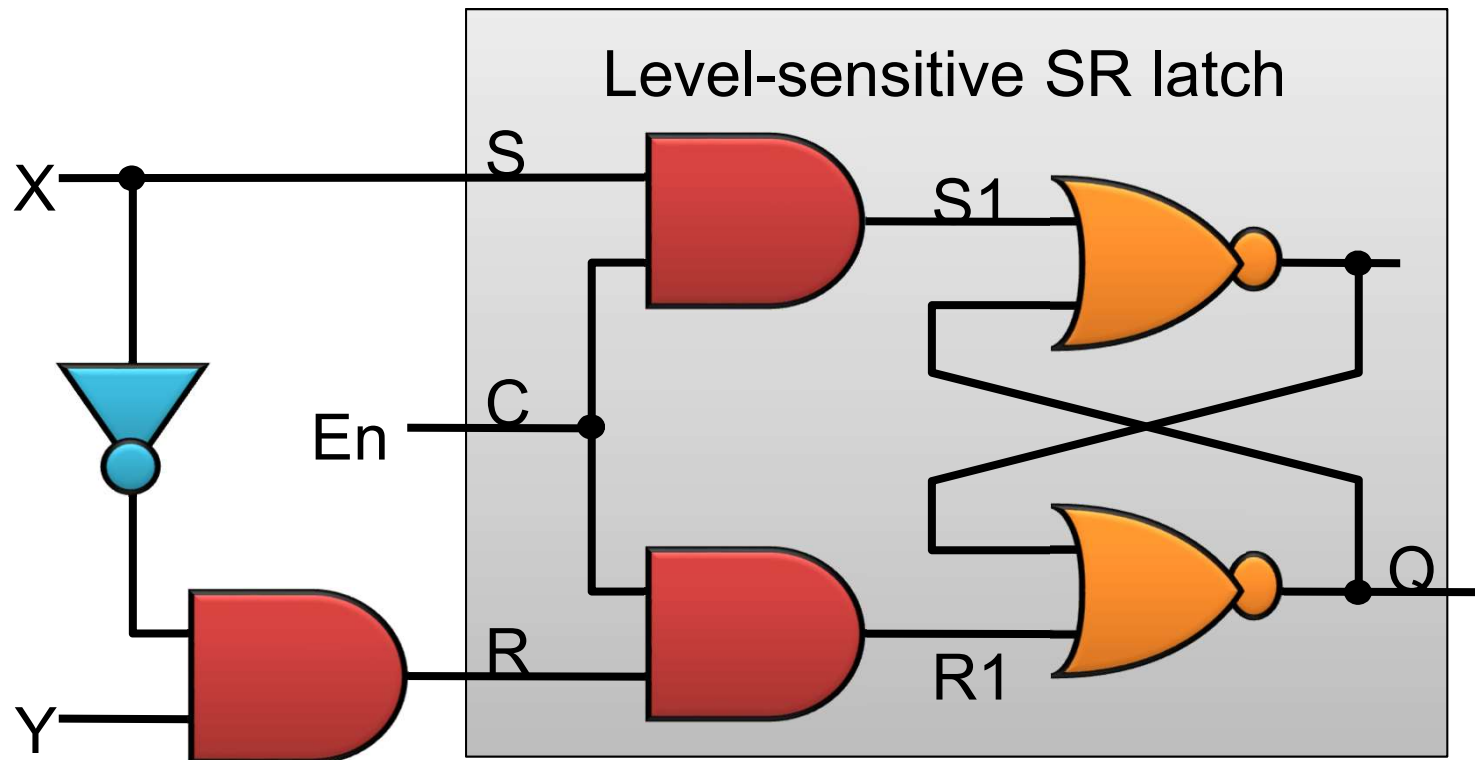


Level-sensitive SR latch symbol

Ensure $S=1$ and $R=1$ should not happened to Level-Sensitive SR Latch



Solution: Ensure, Stabilize, Store



Ensure
Stage

Never Happens
SR=11

Stabilize
Stage

When C=0
Stabilize SR and
Use when C=1

Store
Stage

Store bit

Thanks